

Trace2Skill: 将轨迹局部经验蒸馏为可迁移的智能体技能

Jingwei Ni^{§, *2,3}, Yihao Liu^{§, *4}, Xinpeng Liu^{§, *4}, Yutao Sun^{§, *5}, Mengyu Zhou^{†1}, Pengyu Cheng¹, Dexin Wang¹, Erchao Zhao¹, Xiaoxi Jiang¹ and Guanjun Jiang¹

¹ 阿里巴巴 Qwen 大模型应用团队, ² 苏黎世联邦理工学院, ³ 苏黎世大学, ⁴ 北京大学, ⁵ 浙江大学

*Work done during an internship at Alibaba. †Corresponding author. § 核心贡献者。

大语言模型 (LLM) 智能体日益依赖领域特定技能, 但人工编写此类技能难以扩展, 而仅凭参数化知识生成的技能往往会遗漏关键的操作陷阱。我们提出 Trace2Skill, 该框架通过对智能体经验进行归纳推理, 将大量并行执行轨迹整合到统一的技能目录中。Trace2Skill 既支持深化现有的人工编写技能, 也支持从较弱的 LLM 生成草稿出发创建实用技能。实验表明, Trace2Skill 在办公工作流、数学推理和视觉问答等多个领域均有效。重要的是, 演化所得的技能并非只是对创建它们所用轨迹的记忆产物: 它们通常能够跨模型规模、跨模型家族以及向分布外场景迁移。例如, 由 Qwen3.5-35B 轨迹演化出的技能在 WikiTableQuestions 上可使 Qwen3.5-122B 智能体提升多达 57.65 个百分点。进一步分析表明, Trace2Skill 优于顺序技能编辑和 ReasoningBank 风格的检索记忆, 能够将反复出现的失败与规避方法压缩成标准操作规程 (SoP), 并产出无需参数更新或测试时检索即可复用的可移植技能。^a

^a代码: <https://github.com/Qwen-Applications/Trace2Skill>

1. 引言

基于 LLM 的智能体越来越依赖技能: 编码任务流程、领域知识和操作指南的可复用文档 (Anthropic, 2026d; Zhou et al., 2026b)。随着智能体进入专业化的文件和工具使用 workflow, 瓶颈不再仅是模型能力, 还包括为每个领域创建和维护高质量技能的能力 (Han et al., 2026; Li et al., 2026a; Anthropic, 2026b; Liang et al., 2026; Zhou et al., 2026b)。人工编写的技能可以提供帮助, 但它们并非对所有智能体都同样有益: 在表 1 中, 官方 xlsx 技能提升了 122B 电子表格智能体, 却损害了同一基准上的 35B 智能体。纯粹从参数知识生成技能也很脆弱, 因为这类草案往往缺乏实际执行轨迹所暴露的具体失败模式、规避方法和操作细节 (Li et al., 2026b; Jiang et al., 2026; Zhou et al., 2026b)。

因此, 近期系统使用智能体经验在线演化技能或记忆 (Yang et al., 2026; Xia et al., 2026a; Alzubai et al., 2026; Zhou et al., 2026a; Jiang et al., 2026; Zhou et al., 2026b)。这一方向很有前景, 但许多现有方法要么存储轨迹局部经验以供检索, 要么随着新轨迹到来而顺序编辑技能 (图 1, 左)。这类设计可能将可复用知识分散在庞大的记忆或技能集合中, 而顺序编辑可能使后续技能依赖于早期更新的顺序 (Li, 2026)。人类专家通常会以不同方式工作: 他们检视广泛的轨迹, 抽象出重复出现的模式, 并编写可在不同案例间复用的紧凑流程。

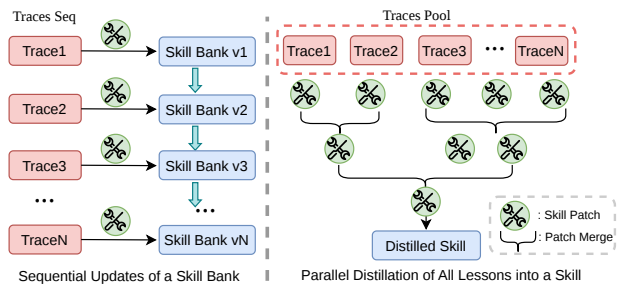


Figure 1 | 左: 顺序在线技能演化会在每条新轨迹到达后编辑技能。右: Trace2Skill 并行分析多条轨迹, 并以层次化方式将反复出现的经验整合为一项可移植技能。

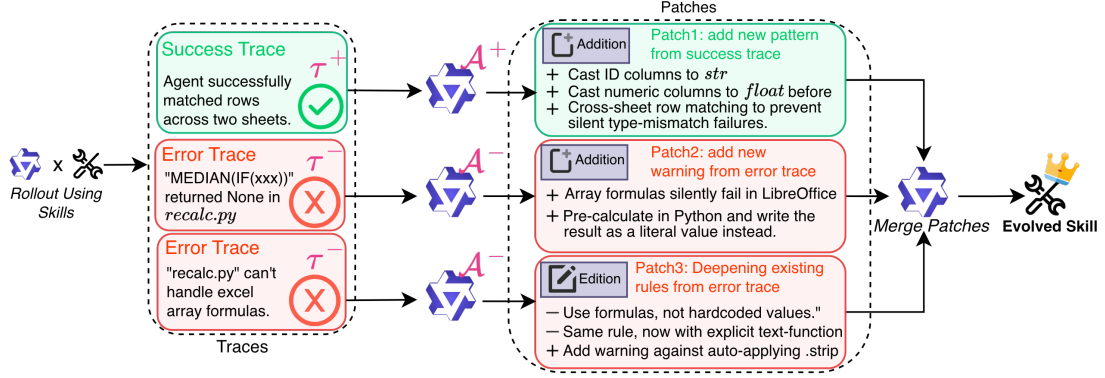


Figure 2 | Trace2Skill 三阶段流水线概览：(1) 运行冻结的智能体以收集带标签的成功与失败轨迹；(2) 使用相互独立的错误与成功分析器，并行提出轨迹级技能补丁；(3) 将所有补丁分层合并为一个可移植的技能目录。

我们提出 Trace2Skill，一个将执行轨迹转化为可移植技能的框架。Trace2Skill 不会在测试时检索每个回合的记忆，也不会通过依赖顺序的编辑吸收轨迹；它联合分析多条轨迹，并将重复经验整合到一个技能目录中（图 1，右）。同一机制支持两种常见用例：深化现有的人工编写技能，以及从较弱的 LLM 生成草案创建实用技能。这种多对一整合对智能体经验执行归纳推理步骤 (Xiong et al., 2025; Li et al., 2025; Lin et al., 2025)，同时保留了普通技能文件的可移植性。

实验表明，具有轨迹根据的技能能在提升性能的同时，保持跨模型、基准和任务领域的可移植性。在电子表格 workflows 中，Trace2Skill 既强化了 Anthropic 官方 xlsx 技能，又从零创建了实用技能。所得技能可在不同模型规模和家族之间迁移：例如，使用 Gemma-4-31B-it (Google, 2026) 演化的技能能够提升 Qwen3.5-122B (Team, 2026)。它们还能泛化到分布外 (OOD) 数据和任务，例如从电子表格编辑迁移到表格问答。收益并不局限于电子表格：Trace2Skill 提升了数学推理和 DocVQA，并进一步强化了 Anthropic 官方的 PDF、DOCX 和 PPTX 工作流程技能。进一步分析表明：并行整合比依赖顺序的技能编辑快得多，且总体更强；单一的蒸馏技能优于 ReasoningBank 式的情节记忆检索 (Ouyang et al., 2026)；智能体式分析员通过检查产物和验证修复来产生更好的补丁。定性上，学到的补丁并非针对特定轨迹的技巧：它们融合成可复用的 SoP，而补丁选择研究表明，其价值往往是组合性的，因而整体整合比贪心局部选择更可靠。

我们的贡献包括：(1) Trace2Skill，一个用于自动创建和深化技能的框架。它模仿人类的技能编写方式：在起草技能之前，通过广泛的轨迹分析构建广泛先验知识 (§ 2)。 (2) 广泛的实证证据，表明具有轨迹根据的演化产生的技能可有效迁移到不同 LLM 规模、家族和 OOD 任务 (§ 3)。 (3) 全面分析，说明整合为何有效：并行合并优于顺序更新，单一整合技能优于基于检索的推理记忆，智能体式诊断提升了补丁质量，且补丁价值往往具有组合性 (§ 4)。

2. Trace2Skill

图 2 展示了 Trace2Skill 的三阶段流水线：收集轨迹，并行提出轨迹级补丁，并将它们整合为一个可移植技能。我们首先定义技能演化目标，然后描述各个阶段。

2.1. 技能与问题形式化

技能是一个人类可读的目录 $\mathcal{S} = (\mathcal{M}, \mathcal{R})$ ，其中 $\mathcal{M}$ 是根 SKILL.md， $\mathcal{R}$ 包含辅助参考、脚本或资产。根文档存储广泛适用的程序性知识，而辅助文件提供确定性工具或较低频的细节。设 π_θ 为推理时使用技能 $\mathcal{S}$ 的固定 LLM 智能体， $\mathcal{P}(\mathcal{S}; \pi_\theta, \mathcal{D})$ 表示该智能体在任务集 $\mathcal{D}$ 上的通过率。给定演化集 $\mathcal{D}_{\text{evolve}}$ 和与之不相交的测试集 $\mathcal{D}_{\text{test}}$ ，技能演化在不更新 θ 的情况下，从演化集轨迹构建新技能：

$$\mathcal{S}^* = \mathcal{E}(\mathcal{S}_0, \mathcal{D}_{\text{evolve}}; \pi_\theta), \quad \mathcal{P}(\mathcal{S}^*; \pi_\theta, \mathcal{D}_{\text{test}}) > \mathcal{P}(\mathcal{S}_0; \pi_\theta, \mathcal{D}_{\text{test}}). \quad (1)$$

我们评估 $\mathcal{S}_0$ 的两种初始化：人类专家技能，以及仅从参数知识生成的 LLM 草案。

2.2. Stage 1: 轨迹生成

我们使用 ReAct 风格的测试框架 (Yao et al., 2023)。对于动态演化集合中的每个任务，固定智能体使用 $\mathcal{S}_0$ 运行，并生成一条轨迹 τ_i ，其中包含查询、推理/工具使用历史、最终输出和二元正确性结果。生成的语料库 $\mathcal{T}$ 被划分为失败轨迹 $\mathcal{T}^-$ 和成功轨迹 $\mathcal{T}^+$ 。轨迹生成可在动态演化集合的问题之间并行进行，因此可以在提出补丁之前独立收集轨迹语料库。提示词模板见 附录 I.1。

2.3. 阶段 2: 并行补丁提议

一组分析员子智能体独立地从各条轨迹中提出技能补丁。失败轨迹被发送给错误分析员 $\mathcal{A}^-$ ，成功轨迹被发送给成功分析员 $\mathcal{A}^+$ ，分析员读取 $\mathcal{S}_0$ 并为其提出补丁，由此形成补丁池 $\mathcal{P} = \mathcal{P}^- \cup \mathcal{P}^+$ 。

分析员的角色被有意设计为非对称形式。 $\mathcal{A}^+$ 使用单遍工作流，从成功轨迹中识别可复用的行为模式。 $\mathcal{A}^-$ 使用 ReAct 式循环，它可以检查轨迹和产物，将输出与标准答案比较，并在提出补丁前验证候选修复。无法得到因果解释的失败会被排除，从而确保 $\mathcal{P}^-$ 以已验证的失败机制为根据，而非仅依赖日志的猜测 (Ouyang et al., 2026)。我们提示两种角色按照技能编写指南 (Anthropic, 2026b) 编写简洁且可操作的补丁。分析员提示模板和具有代表性的补丁示例见 附录 I.2。

2.4. Stage 3: 补丁整合

阶段 3 将补丁池整合为一项连贯的更新 p^* ，并将其应用于 $\mathcal{S}_0$ 。补丁会经过 $L = \lceil \log_{B_{\text{merge}}} |\mathcal{P}| \rceil$ 层的分层合并；在每一层 ℓ ，最多将 B_{merge} 个补丁综合为一个补丁：

$$p^{(\ell+1)} = \mathcal{M}\left(\pi_\theta, \mathcal{S}_0, \{p_1^{(\ell)}, \dots, p_{B_{\text{merge}}}^{(\ell)}\}\right), \quad \ell = 0, \dots, L-1, \quad (2)$$

其中， $\mathcal{M}$ 会去除重复内容、解决冲突并保留互不重叠的见解。 π_θ 本身同时充当轨迹生成器、分析器和合并算子，因此不需要外部演化/教师模型。最终的 p^* 会被转换为差异格式的编辑，并在确定性保护机制下应用：拒绝编辑不存在的文件、搁置行范围冲突，并验证更新后的技能格式。

合并过程还会进行归纳泛化。由于每个补丁均来自一条轨迹，独立补丁中反复出现的编辑表明存在系统性的任务属性，而非一次性修复。因此， $\mathcal{M}$ 被要求优先选择常见模式并舍弃特殊性编辑，生成一项紧凑技能以替代 $\mathcal{S}_0$ ，并在推理时直接使用，无需检索。合并算子的提示词模板和一个整合补丁示例 p^* 见 附录 I.3。

Trace2Skill 支持两种模式，分别覆盖有无专家编写技能的场景：技能深化从人工编写的技能开始，技能创建则从由 LLM 参数化知识生成的技能开始。两种模式都利用轨迹分析归纳出的补丁改进目标技能。

Table 1 | 不同技能作者模型（负责演化技能）和技能使用者模型（推理时运行技能）下的主要电子表格结果。SpreadsheetBench 报告 Vrf（已验证）、Soft（问题通过率）和 Hard（任务通过率）；WikiTQ 与 HiTab 表示分布外（OOD）表格问答迁移， $Avg=(ID_{avg}+OOD_{avg})/2$ 对分布内（ID, SpreadsheetBench）和 OOD 等权。参考行是绝对分数；演化行是相对人工编写（深化）或参数化（创建）基线的带符号增量。每列最大绝对分数以**粗体**标出。各随机种子的标准差见 表 7。

Condition	Skill User: Qwen3.5-122B-A10B					Skill User: Qwen3.5-35B-A3B					
	SpreadsheetBench			OOD		SpreadsheetBench			OOD		Avg↑
	Vrf↑	Soft↑	Hard↑	WikiTQ↑	HiTab↑	Vrf↑	Soft↑	Hard↑	WikiTQ↑	HiTab↑	
Baseline (absolute scores)											
No Skill	27.67	28.90	17.57	21.50	14.42	19.00	18.00	4.60	13.33	19.12	18.19
Human-Written	48.33	36.30	17.03	74.68	41.31	9.67	13.03	3.37	9.02	5.30	26.93
Parametric	26.17	36.60	17.50	23.73	17.36	20.17	13.70	3.87	20.14	13.94	19.23
Skill Author: Qwen3.5-122B-A10B											
深化（相对人工编写初始技能的增量）											
+Error	+17.50	+10.30	+10.40	+1.62	+2.14	+27.00	+9.44	+2.86	+9.26	+9.55	+9.28
+Success	+18.00	+8.60	+8.70	-10.35	-0.95	+19.66	+6.84	+1.33	+12.09	+22.33	+8.15
+Combined	+21.50	+10.87	+12.50	+4.56	+2.97	+21.16	+8.84	+1.80	+6.64	+11.94	+9.65
创建（相对参数化初始技能的增量）											
+Error	+22.83	+3.77	+5.87	+7.89	-1.70	+8.66	+9.53	+4.00	+2.06	+9.64	+6.79
+Success	+15.33	-0.93	+4.33	+23.70	+19.89	+12.83	+11.57	+6.13	+30.36	+28.90	+16.96
+Combined	+14.00	-0.63	+3.53	+32.32	+23.11	+15.50	+14.50	+7.23	+29.70	+27.71	+18.62
Skill Author: Qwen3.5-35B-A3B											
深化（相对人工编写初始技能的增量）											
+Error	+16.67	+8.50	+8.14	-6.36	-2.38	+17.33	+9.17	+4.83	+2.71	+8.08	+5.64
+Success	+2.17	+2.73	+3.30	+1.46	+1.70	+12.33	+5.87	+1.23	+43.23	+34.70	+12.44
+Combined	+6.67	+3.87	+4.17	+2.65	+2.44	+20.00	+5.77	+2.36	+42.20	+36.46	+14.04
创建（相对参数化初始技能的增量）											
+Error	+1.00	-7.70	+1.03	+57.65	+28.25	+3.83	+7.30	+2.66	+12.66	+17.81	+15.22
+Success	+5.33	-4.57	+2.43	+9.09	+2.34	+5.66	+5.80	+2.63	+3.31	+18.94	+5.65
+Combined	+8.33	-5.83	+2.00	+30.82	+16.93	+4.33	+9.73	+4.73	+18.00	+25.22	+13.31

3. 实验

3.1. 实验设置

电子表格设置。我们的主要实验使用 SpreadsheetBench-Verified (Ma et al., 2024)，其中智能体通过工具使用来操作 xlsx 文件。我们将其 400 个样本划分为 200 个演化问题和 200 个留出测试问题；此外还报告完整 SpreadsheetBench 上的 Soft/Hard 分数（排除演化集中的所有问题），以及向转换为电子表格格式的 WikiTableQuestions (Pasupat & Liang, 2015) 和 HiTab (Cheng et al., 2022) 的 OOD 迁移结果。所有电子表格结果均使用各基准的官方评估标准，并在三个随机种子上取平均。

技能设置。我们比较 No Skill、Anthropic 的 xlsx 技能 (Human-Written)、使用**参数化**知识生成的 LLM 技能，

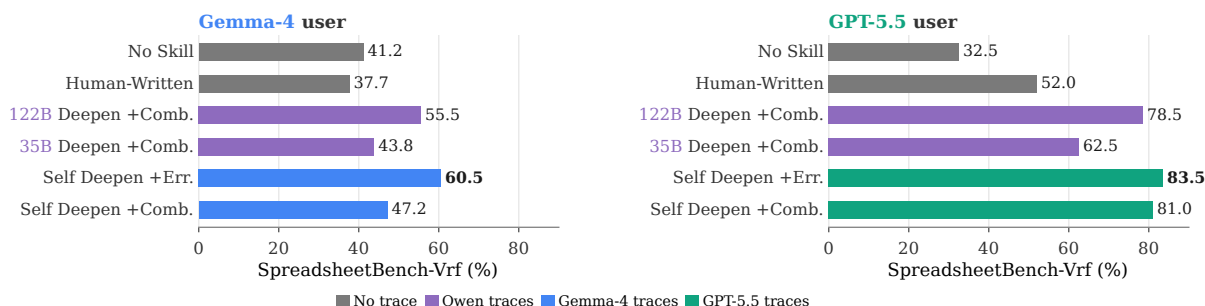


Figure 3 | SpreadsheetBench 上的跨模型族泛化。Gemma-4-31B-it 和 GPT-5.5-high 各自：(i) 从自身轨迹演化技能；(ii) 运行由 Qwen3.5 创作的技能。

以及 Trace2Skill 的三个变体：+Error、+Success 和 +Combined，它们分别仅整合失败轨迹、仅整合成功轨迹以及整合全部轨迹产生的补丁。Skill Deepening 从 Human-Written 开始，而 Skill Creation 从 Parametric 开始。我们评估 Qwen3.5-122B-A10B 和 Qwen3.5-35B-A3B，让它们同时作为技能编写者和技能使用者。我们进行 100% 自我演化：同一模型生成轨迹、提出补丁并编辑技能。数据集构建、评分和模型部署的细节见附录 A；外部 skill-creator 基线见附录 H。

3.2. 主要结果

表 1 报告了所有电子表格结果，涵盖各种技能条件、作者模型、用户模型和迁移任务。基线行给出绝对分数；演化行给出相对相应基线的带符号变化量，其中 Deepening 与 Human-Written 比较，Creation 与 Parametric 比较。我们使用 Avg 作为主要汇总指标，因为它对两个用户模型的 ID 和 OOD 性能赋予相同权重，从而奖励能够跨模型和任务泛化的技能。

基线揭示了两种设置为何都很重要。 Human-Written 是一种有用的人工先验，但无法在不同模型规模间可靠迁移；Parametric 仍接近 No Skill，证实仅有参数化知识无法产生可执行的电子表格技能 (Han et al., 2026)。这些基线促成了两种评估范式：Deepening 测试强大的人工技能能否变得更具迁移性，而 Creation 测试基于轨迹的蒸馏能否从弱种子构建有用技能。

Deepening 和 Creation 都能产生可泛化技能。 从 Human-Written 开始，演化后的技能持续增强分布内电子表格性能，并常常迁移至其他模型规模和 OOD 表格任务；从 Parametric 开始，Creation 在有利设置下可以达到或超过 Human-Written 的质量。这些增益十分广泛：不仅出现在生成轨迹的模型上，还横跨各种作者模型、用户模型和任务族。值得注意的是，35B Deepening +Combined 达到了最佳绝对 Avg，说明小型 LLM 创作的技能同样能够泛化。在各种分析器中，+Error 和 +Success 都有助于电子表格任务，而 +Combined 通常带来最大的 Avg 提升。

3.3. 模型族泛化

我们使用 Gemma-4-31B-it 和 GPT-5.5-high，在两种设置下测试跨模型族泛化：每个模型 (i) 从自身轨迹出发深化人工编写的 xlsx 技能，所采用的方法是 Trace2Skill；(ii) 运行 Qwen3.5-122B/35B +Combined 深化技能。图 3 显示，这两个模型既能从自身轨迹中实现自我改进，也能受益于 Qwen3.5 创作的技能，说明 Trace2Skill 可以跨模型族泛化。附录 B.4 报告了其他结果与实现细节。

Table 2 | 向留出 DAPO 与 AIME 的数学迁移。D-Test 表示 DAPO-Math-Test 通过率；AIME 使用 avg@8。

Condition	Skill User: 122B		Skill User: 35B	
	D-Test↑	AIME↑	D-Test↑	AIME↑
Reference (absolute scores)				
No Skill	91.0	90.4	89.0	83.3
Skill Author: Qwen3.5-122B-A10B				
创建 (初始: 无技能)				
+Error	+4.0	+2.9	+5.0	+5.0
+Success	+2.0	+0.4	+2.0	+4.6
+Combined	+3.0	-1.2	+6.0	+5.5
Skill Author: Qwen3.5-35B-A3B				
创建 (初始: 无技能)				
+Error	+3.0	+1.3	+4.0	+0.5
+Success	-1.0	-1.2	+0.0	-1.2
+Combined	+1.0	-0.4	+1.0	+0.0

3.4. 数学推理

我们将 Trace2Skill 应用于数学领域，以检验其领域无关性。与电子表格设置 (§ 3.1) 一样，数学智能体在 ReAct 循环中运行，并使用命令行 Python 解释器为每个问题编写和执行代码。我们在 400 个 DAPO 问题上从零创建技能 (Yu et al., 2025)，并按照 § 3.2 的跨模型协议，在 100 个互不重叠的留出 DAPO 问题 (ID) 和 AIME 2026 (OOD 竞赛数学; 30 道题的 avg@8) 上进行评估。表 2 报告了相对 No Skill 的变化量。Trace2Skill **适用于数学领域**：蒸馏出的技能同时改善了留出的 DAPO 和 OOD AIME，而非过拟合源分布。**+Error 是最稳定的信号**，可以在 122B 和 35B 用户模型之间实现稳定迁移。

3.5. 视觉问答

为测试多模态泛化能力，我们将 Trace2Skill 应用于 DocVQA (Mathew et al., 2020)，其中智能体需要回答有关表单、表格、发票、信件和报告等文档图像的问题。同样，每个智能体都在具有命令行 + Python 环境的 ReAct 循环中运行。我们仅使用 50 个 DocVQA 样例进行技能演化，并将其从评估中移除；其余 5,299 个样例构成留出测试集。对于同模型使用以及 122B 与 35B 编写的技能之间的跨模型迁移，我们报告官方 ANLS 和 Accuracy (ANLS ≥ 0.5 , %)。表 3 中的结果表明，**所有经演化的 DocVQA 技能相较 No Skill 均提升了性能**。**+Combined 是最强的设置**，它带来了最明显的同模型收益和正向的跨模型迁移。

4. 分析

本节分析 Trace2Skill 奏效的原因及其更广泛的应用。我们首先在共享轨迹池和执行框架下剥离并考察其三个核心设计选择 (§ 4.1)，随后刻画其提炼出的标准操作规程 (§ 4.2)，研究补丁价值如何组合 (§ 4.3)，并在更广泛的真实世界任务中检验迁移能力 (§ 4.4)。

Table 3 | 由轨迹蒸馏的视觉推理技能向 DocVQA 迁移。ANLS 表示相似度；Acc 表示 ANLS ≥ 0.5 。

Condition	Skill User: 122B		Skill User: 35B	
	ANLS↑	Acc↑	ANLS↑	Acc↑
Reference (absolute scores)				
No Skill	0.6300	70.27	0.6582	73.17
Skill Author: Qwen3.5-122B-A10B				
创建 (初始: 无技能)				
+Error	+0.1949	+17.69	+0.1974	+16.64
+Success	+0.1639	+14.89	+0.1668	+14.23
+Combined	+0.2534	+22.25	+0.2049	+17.22
Skill Author: Qwen3.5-35B-A3B				
创建 (初始: 无技能)				
+Error	+0.0884	+6.79	+0.1267	+10.15
+Success	+0.1572	+14.53	+0.2132	+18.27
+Combined	+0.0958	+7.73	+0.2158	+18.83

4.1. 核心设计比较

Trace2Skill 基于三项设计选择。在此，我们使用相同的轨迹池和执行框架，将每项选择与其自然替代方案进行比较，从而分别研究：并行多对一整合与顺序编辑（在不损失质量的前提下提速）、单一整合技能与经验记忆检索（无需测试时检索的复用），以及智能体式与单次调用式错误分析（更准确的根因补丁）。

Table 4 | SpreadsheetBench 上的并行整合与顺序编辑对比（相同轨迹池，+Error）。并行方法在效果和效率上均优于顺序方法。表 8 与 表 10 展示了数学/VQA 上的类似趋势。

Condition	Skill User: 122B			Skill User: 35B			Time↓
	Vrf↑	Soft↑	Hard↑	Vrf↑	Soft↑	Hard↑	
Seq-B=4	59.00	40.63	20.63	26.17	22.37	7.47	~15 min
Seq-B=1	61.83	44.40	25.40	26.00	23.83	10.57	~60 min
Parallel (ours)	65.83	46.60	27.43	27.00	22.20	8.20	~3 min

并行整合。在线技能演化通常会随着轨迹批次的到来编辑技能，因此后续分析会依赖于之前的编辑。我们将并行多对一整合与两个顺序基线进行比较，以分离其效果：Seq-B=1（每条轨迹后更新）和 Seq-B=4（每四条轨迹后更新）。所有条件均仅使用错误分析员，并从 Human-Written 技能初始化。表 4 报告了由此得到的质量与墙钟时间权衡。

并行整合在 122B SpreadsheetBench 的所有指标以及 35B Vrf 上都更好；唯一例外是 Seq-B=1 在 35B Soft 和 Hard 上略高。这一小幅质量权衡伴随着巨大的效率差距：并行方式约需 3 min 即可产生技能，而 Seq-B=4 和 Seq-B=1 分别需要 15 min 和 60 min。数学推理和 DocVQA 也呈现相同的总体趋势：并行整合在保持或提升质量的同时速度快得多；附录 B 给出了延迟分析以及在数学/VQA 上的类似比较。

整体技能与检索。过往经验检索是复用智能体轨迹的常见范式：经验记忆系统存储来自之前执行的反思、记忆或程序性经验，并在推理时检索相关条目 (Shinn et al., 2023; Wang et al., 2023a; Ouyang et al., 2026; Fang et al., 2026; Wang et al., 2024b; Liu et al., 2025)。

我们使用 ReasoningBank (Ouyang et al., 2026) 实例化这一范式：遵循其原始设置，我们存储成功与失败轨迹中的经验，并使用 Qwen3-Embedding-8B 检索排名第一的记忆。我们将其与使用相同轨迹池的 +Combined 进行比较。相同模型 Deepening 的结果见 表 5。+Combined 始终优于 ReasoningBank。这表明，相比在测试时检索孤立记忆，将轨迹证据整合为紧凑技能更具优势。我们不进行 OOD 评估，因为 WikiTQ 和 HiTab 的查询与 SpreadsheetBench 的查询在语义上相距过远，不适合采用检索方法。

Table 5 | 在 SpreadsheetBench 上，Trace2Skill 优于 ReasoningBank 风格的检索（轨迹池相同）。表 9 与 表 11 展示了数学/VQA 上的类似趋势。

Setting	User	Vrf↑	Soft↑	Hard↑
ReasoningBank	122B	56.00	40.10	21.30
(Ouyang et al., 2026)	35B	20.50	17.30	4.97
Human-Written	122B	69.83	47.17	29.53
+Combined (ours)	35B	29.67	18.80	5.73

智能体式错误分析。相关工作通过单次非交互式 LLM 调用，从错误轨迹中提炼可迁移的经验或技能 (Ouyang et al., 2026; Xia et al., 2026a; Yang et al., 2026; Jiang et al., 2026)。我们通过 +Error LLM 对这一设计进行消融：一次 LLM 调用读取每条失败轨迹并提出补丁，但不检查产物、不查询真实答案，也不验证修复；图 4 将其与我们的智能体式错误分析器进行比较。在大多数设置下，Agentic +Error 的 Avg 均高于 +Error LLM，

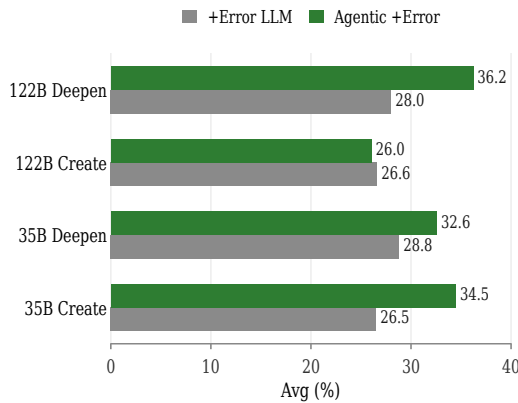


Figure 4 | Agentic +Error 与单次调用 +Error LLM 在 Avg (ID 与 OOD 性能的平衡, 与表 1 中的 Avg 相同) 上的比较。智能体式分析器会检查产物并验证修复, 而 +Error LLM 仅阅读执行日志。



Figure 5 | SpreadsheetBench-Vrf 上的贪心选择性补丁聚合 (通过率, %)。每条曲线在每次迭代 (x 轴) 中从 Error、Success 或 Combined 池加入一个经验证集选出的补丁; 水平线表示对全部补丁进行完整的 Trace2Skill 聚合。

说明交互式诊断比仅根据日志生成补丁更有效; 附录 D 报告并讨论了表 12 中按数据集划分的完整结果。附录 C.1 中的定性分析揭示了其机制: 访问产物并验证修复, 有助于智能体式分析器更准确地识别根因。

同条件比较与 正面对比。上述三项比较均为同条件比较, 在排除数据、模型、测试框架和工程因素这些会使完整系统评估复杂化的混杂因素后, 分别考察每个核心设计。作为补充视角, 附录 G 报告了在共享开源模型和基准下, 与三个同期系统 (XSkill (Jiang et al., 2026)、EvoSkill (Alzubi et al., 2026) 和 SkillGen (Ma et al., 2026)) 的直接正面对比, 其中 Trace2Skill 仍表现出优势。

4.2. 学习到的 SoP

多对一合并保留跨轨迹反复出现的操作, 并将其转化为标准操作规程 (SoP), 而不是收集一次性技巧。在 122B Deepening +Combined 运行产生的 323 个映射补丁中, 四种学得的 SoP 占据主导地位, 每种分别被 16% 至 55% 的补丁引用 (单个补丁可以引用多个 SoP), 其中包括: 写入后重新计算并回读公式、提交前核验目标单元格, 以及按照不会造成数据损坏的顺序删除行。任务特有的细节被转入按需使用的 references/ 文件, 使 SKILL.md 专注于可广泛复用的规程 (详见附录 C.3)。

4.3. 选择性补丁聚合

Trace2Skill 将所有学习到的补丁聚合进目标技能, 避免逐补丁验证的成本。如果补丁质量参差不齐, 选择一个子集是否会更好? 我们将两种选择器与完整聚合进行比较: **贪心** top-1 在每次迭代中依据在动态演化集合上的验证结果, 加入局部最优补丁; **贝叶斯优化** (BO) 搜索二进制补丁纳入向量, 并复用此前尝试过的子集的验证分数, 使后续提案偏向有前景的组合。两者都使用包含 200 个任务的动态演化集合 (§ 3.2), 以及从该集合中抽取的 32 个任务作为验证集; 完整算法细节见附录 E。

贪心方法上升很快, 但在低于完整聚合的水平进入平台期。图 5 显示, 贪心 +Error 和 +Combined 在前几次迭代中有所上升, 随后进入平台期, 而 +Success 则先下降、之后仅部分恢复; 所有贪心曲线都低于完整的 Trace2Skill 聚合。造成平台期的机制有两个 (详见附录 C.2)。(1) 补丁无关回退: 补丁使部分目标任务由错误变为正确, 但其副作用也使此前正确的任务由正确变为错误, 因而净准确率停滞。(2) 语义重叠: 反复出现

Table 6 | 在 122B Deepening 电子表格设置中, BO 选择的 +Error 补丁与应用全部补丁 (No Selection) 的比较。Evolve 是在演化集上的准确率。

技能	Evolve	Vrf	Soft	Hard	WikiTQ	HiTab
No Selection (+Error)	68.00	65.83	46.60	27.43	76.30	43.45
BO Selection (+Error)	72.83	69.83	47.27	28.90	77.51	43.38

的错误促使流水线不断提出针对同类行为的补丁 (例如公式重算、验证清单), 因此新补丁很大程度上只是重述已有指导, 几乎不带来边际增益。因此, 补丁价值是组合性的, 而不是单个补丁价值的简单求和, 这使得优化补丁的组合效果成为必要。

BO 有效, 但计算开销很大。 BO 直接搜索补丁子集, 因此能够刻画贪心方法遗漏的互补与干扰关系。表 6 显示, 与应用全部 +Error 补丁相比, BO 改善了大多数选定指标, 尤其是 SpreadsheetBench 各列。代价在于验证: 每个候选子集都必须被具体化为一项技能, 并在验证集上运行, 之后才能为后验分布提供信息; 随着补丁全集增大, 这一成本会迅速上升。因此, 我们认为 BO 适用于目标分布和验证预算已知的情形, 而非用于取代 Trace2Skill 中默认的完整聚合。

4.4. 更广泛的应用

我们在 PDF 提取、PPTX 编辑和 DOCX 编辑中测试 Trace2Skill 的更广泛应用。这些设置以 Anthropic 官方的 pdf、pptx 和 docx 技能为起点 (Anthropic, 2026a), 并沿用与电子表格实验相同的执行方式: 智能体使用命令行工具操作文件, 并由任务专用的本地验证器评分。

在这三个领域中, 基于源轨迹的演化都提升了独立留出目标上的性能。对于 PDF, VRDU 轨迹可以迁移到 VAREX, 使通过率从 76.9% 提升至 85.3% (Wang et al., 2023b; Barzelay et al., 2026)。对于 PPTX, TSBench 轨迹可以迁移到幻灯片组不重叠的 TSBench OOD 划分, 使性能从 72.5% 提升至 88.8% (Jung et al., 2026)。对于 DOCX, 在涵盖通用 DOCX 操作的合成任务上进行演化, 可以迁移到 OfficeBench 的 DOCX 子集, 使性能从 79.7% 提升至 87.5% (Wang et al., 2024a)。附录 F 给出了各领域的设置与实现细节。

5. 相关工作

智能体技能。 智能体技能将任务流程、领域知识和操作保护机制封装为可加载的产物 (Anthropic, 2026d), 但生态系统和基准表明, 这种抽象需要谨慎设计: 聚焦且匹配良好的技能可提升性能, 而宽泛、陈旧或不匹配的技能可能分散智能体注意力, 甚至损害其表现 (Zhou et al., 2026b; Li et al., 2026b; Han et al., 2026; Li, 2026; Li et al., 2026a; Liang et al., 2026)。Trace2Skill 延续了将技能视为可移植 SoP 的观点, 但提出了一个更聚焦的问题: 如何将广泛的执行证据压缩为能够在不同模型和任务分布间持续发挥作用的指导。

用于智能体自我演化的经验记忆。 另一类方法通过存储执行经验以供复用来改进智能体, 涵盖从语言反思和累积行为, 到在测试时查询的检索式、程序式、工作流式或回放式记忆 (Shinn et al., 2023; Wang et al., 2023a; Ouyang et al., 2026; Fang et al., 2026; Wang et al., 2024b; Qian et al., 2024; Nottingham et al., 2024; Liu et al., 2025)。这些方法都认同我们的前提, 即经验中包含可复用的结构, 但它们会保留外部记忆或检索模块; 与之不同, Trace2Skill 将大量局部观察蒸馏到一个静态技能目录中。它偏向归纳压缩, 而非最近邻复用, 并避免在测试时依赖检索质量。

技能与策略演化。 与我们最接近的同期工作通过智能体交互或引导式优化来演化技能 (Zheng et al., 2025;

Yang et al., 2026; Jiang et al., 2026; Alzubi et al., 2026; Zhou et al., 2026a; Ma et al., 2026; Anthropic, 2026b), 而范围更广的一类工作则通过技能增强强化学习、内在技能演化或元学习, 在线协同演化策略、技能或智能体循环 (Xia et al., 2026a; Li et al., 2026c; Xia et al., 2026b)。Trace2Skill 探讨了一个互补问题: 使用一个模型收集的轨迹, 能否被压缩到一个直接有益于其他模型和任务的静态技能目录。它不维护测试时记忆、检索索引或更新后的策略, 而是将轨迹局部证据整合为可原样加载的可复用 SoP。因此, 我们关注的不是特定智能体循环内的自适应复用, 而是学习产物在不同部署设置间的可移植性。

6. 结论

我们提出了 Trace2Skill, 这是一种将智能体执行轨迹提炼为可移植技能的框架: 并行分析子智能体从互不重叠的轨迹批次中提出针对性补丁, 随后通过整合步骤一次性将其合并为一个声明式技能目录。从单一模型轨迹中提炼出的技能可以跨模型规模、跨模型家族以及向 OOD 任务迁移。Trace2Skill 在多个领域中也表现出很强的适用性。

局限性

Trace2Skill 默认应用全部整合补丁; 使用贝叶斯优化选择质量更高的子集可能有所帮助 (§ 4.3), 但代价高昂。可靠的选择信号需要规模大且具有代表性的验证集, 而 BO 必须为每个候选子集实际生成并评估一项新技能, 因此成本会随验证集规模和补丁全集大小迅速增长。理想情况下, 使用整个演化集进行验证会比我们仅含 32 个问题的抽样验证更准确地反映补丁质量, 但计算成本也会高得多。因此, 我们将对组合式补丁选择的更深入探索留待未来工作。

伦理声明

我们使用公开可用的数据集, 不涉及数据隐私问题。我们所用的所有产物均采用允许研究使用的许可证。定性分析中的人工标注由本文作者完成。我们未发现本研究涉及任何其他伦理风险。我们使用 AI 编程工具 (Codex/Claude Code) 协助编程, 并确认所有相关编程工作均在严格的人工监督下进行。我们实现了单元测试, 以避免投机取巧或非对齐行为。我们还将开源代码, 以实现完整可复现性。此外, 我们使用 ChatGPT 检查并修正语法, 整个过程完全由人类作者监督。

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems, 2026. URL <https://arxiv.org/abs/2603.02766>.
- Anthropic. PDF, DOCX, and PPTX document skills. GitHub repository, 2026a. URL <https://github.com/anthropics/skills/tree/main/skills>. Accessed: 2026-05-01; document-skill subfolders: pdf, docx, and pptx.
- Anthropic. How to create a skill with claude through conversation. Claude Tutorials, 2026b. URL <https://claude.com/resources/tutorials/how-to-create-a-skill-with-claude-through-conversation>.
- Anthropic. skill-creator. GitHub repository, 2026c. URL <https://github.com/anthropics/skills/tree/main/skills/skill-creator>.
- Anthropic. What are skills? Claude Help Center, 2026d. URL <https://support.claude.com/en/articles/12512176-what-are-skills>. Access Date: 2026-03-22.

- Udi Barzelay, Ophir Azulai, Inbar Shapira, Idan Friedman, Foad Abo Dahood, Madison Lee, and Abraham Daniels. VAREX: A benchmark for multi-modal structured extraction from documents, 2026. URL <https://arxiv.org/abs/2603.15118>.
- Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. HiTab: A hierarchical table dataset for question answering and natural language generation. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1094–1110, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.78. URL <https://aclanthology.org/2022.acl-long.78/>.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL <https://arxiv.org/abs/2508.06433>.
- Google. Gemma 4: Byte for byte, the most capable open models. <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>, 2026. Accessed: 2026-05-18.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering?, 2026. URL <https://arxiv.org/abs/2603.15401>.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. Xskill: Continual learning from experience and skills in multimodal agents, 2026. URL <https://arxiv.org/abs/2603.12056>.
- Kyudan Jung, Hojun Cho, Jooyeol Yun, Soyoung Yang, Jaehyeok Jang, and Jaegul Choo. Talk to your slides: High-efficiency slide editing via language-driven structured data manipulation, 2026. URL <https://arxiv.org/abs/2505.11604>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026a. URL <https://arxiv.org/abs/2603.02176>.
- Jiachun Li, Pengfei Cao, Zhuoran Jin, Yubo Chen, Kang Liu, and Jun Zhao. Mirage: Evaluating and explaining inductive reasoning process in language models, 2025. URL <https://arxiv.org/abs/2410.09542>.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026b. URL <https://arxiv.org/abs/2602.12670>.
- Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail, 2026. URL <https://arxiv.org/abs/2601.04748>.

- Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning, 2026c. URL <https://arxiv.org/abs/2603.16060>.
- Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. Skillnet: Create, evaluate, and connect ai skills, 2026. URL <https://arxiv.org/abs/2603.04448>.
- Brian S. Lin, Jiaxin Yuan, Zihan Zhou, Shouli Wang, Shuo Wang, Cunliang Kong, Qi Shi, Yuxuan Li, Liner Yang, Zhiyuan Liu, and Maosong Sun. On llm-based scientific inductive reasoning beyond equations, 2025. URL <https://arxiv.org/abs/2509.16226>.
- Yitao Liu, Chenglei Si, Karthik Narasimhan, and Shunyu Yao. Contextual experience replay for self-improvement of language agents, 2025. URL <https://arxiv.org/abs/2506.06698>.
- Yuchen Ma, Yue Huang, Han Bao, Haomin Zhuang, Swadheen Shukla, Michel Galley, Xiangliang Zhang, and Stefan Feuerriegel. Skillgen: Verified inference-time agent skill synthesis, 2026. URL <https://arxiv.org/abs/2605.10999>.
- Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation, 2024. URL <https://arxiv.org/abs/2406.14991>.
- Minesh Mathew, Dimosthenis Karatzas, R Manmatha, and CV Jawahar. Docvqa: A dataset for vqa on document images. corr abs/2007.00398 (2020). arXiv preprint arXiv:2007.00398, 2020.
- Kolby Nottingham, Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Sameer Singh, Peter Clark, and Roy Fox. Skill set optimization: Reinforcing language model behavior via transferable skills, 2024. URL <https://arxiv.org/abs/2402.03244>.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In The Fourteenth International Conference on Learning Representations, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables, 2015. URL <https://arxiv.org/abs/1508.00305>.
- Cheng Qian, Shihao Liang, Yujia Qin, Yining Ye, Xin Cong, Yankai Lin, Yesai Wu, Zhiyuan Liu, and Maosong Sun. Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution, 2024. URL <https://arxiv.org/abs/2401.13996>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.

- Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023a. URL <https://arxiv.org/abs/2305.16291>.
- Zilong Wang, Yichao Zhou, Wei Wei, Chen-Yu Lee, and Sandeep Tata. VRDU: A benchmark for visually-rich document understanding. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023b. doi: 10.1145/3580305.3599929. URL <https://arxiv.org/abs/2211.15421>.
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. OfficeBench: Benchmarking language agents across multiple applications for office automation, 2024a. URL <https://arxiv.org/abs/2407.19056>.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024b. URL <https://arxiv.org/abs/2409.07429>.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026a. URL <https://arxiv.org/abs/2602.08234>.
- Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, Siwei Han, Shi Qiu, Haonian Ji, Yuyin Zhou, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Metaclaw: Just talk – an agent that meta-learns and evolves in the wild, 2026b. URL <https://arxiv.org/abs/2603.17187>.
- Chenfei Xiong, Jingwei Ni, Yu Fan, Vilém Zouhar, Donya Rooein, Lorena Calvo-Bartolomé, Alexander Hoyle, Zhijing Jin, Mrinmaya Sachan, Markus Leippold, Dirk Hovy, Mennatallah El-Assady, and Elliott Ash. Co-DETECT: Collaborative discovery of edge cases in text classification. In Ivan Habernal, Peter Schulam, and Jörg Tiedemann (eds.), Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 354–364, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-334-0. doi: 10.18653/v1/2025.emnlp-demos.25. URL <https://aclanthology.org/2025.emnlp-demos.25/>.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution, 2026. URL <https://arxiv.org/abs/2603.01145>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>.

Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. Memento-skills: Let agents design agents, 2026a. URL <https://arxiv.org/abs/2603.18743>.

Yingli Zhou, Shu Wang, Yaodong Su, Wenchuan Du, Yixiang Fang, and Xuemin Lin. A comprehensive survey on agent skills: Taxonomy, techniques, and applications, 2026b. URL <https://arxiv.org/abs/2605.07358>.

A. 实验细节

随机种子与计算资源。除非另有说明，所有报告结果均为三个随机种子 41、42 和 43 上的平均值。实验在配备 8 块 NVIDIA A100 GPU 的节点上运行。所有实验与探索总计约耗费 20,000 GPU 小时。

电子表格数据与评分。SpreadsheetBench Verified 被划分为 200 个演化问题和 200 个留出测试问题；技能演化期间不使用任何留出样本。我们还在完整 SpreadsheetBench 上报告 Soft（子问题通过率）和 Hard（所有子问题都必须通过）；我们从中移除了每个演化集问题，因而技能演化期间见过的问题不会在测试时评分。对于 OOD 迁移，WikiTableQuestions 被转换为基于组合式维基百科表格的电子表格任务，HiTab 则被转换为需要层次索引以及隐式计算和语义推理的电子表格任务。评估集包含 200 个 SpreadsheetBench-Verified 留出样例、2,529 个用于 Soft/Hard 的完整 SpreadsheetBench 样例（已移除所有演化集问题）、2,810 个 WikiTableQuestions 样例和 1,585 个 HiTab 样例。

基线技能。Human-Written 是 Anthropic 官方的 xlsx 技能，Parametric 则是通过仅基于参数知识提示 Qwen3.5-122B-A10B 生成的 xlsx-basic 种子，不具备轨迹根据。

模型服务与演化协议。两个主要的作者/用户模型是 Qwen3.5-122B-A10B 和 Qwen3.5-35B-A3B。我们使用 Hugging Face 检查点 [Qwen/Qwen3.5-122B-A10B](#) 和 [Qwen/Qwen3.5-35B-A3B](#)。两者都是指令/思考混合 MoE 模型：我们对多轮 ReAct 式智能体使用指令模式，对层次合并、成功分析和补丁转换等单次调用步骤使用思考模式。模型通过 vLLM (Kwon et al., 2023) 提供服务；对于推理模式调用，我们遵循 Qwen 官方推荐的解码设置。Stage 1 为每个问题生成一条轨迹；Stage 2 并行运行 128 个子智能体，合并批大小为 32。我们不另行设置工具调用预算。

B. 补充分析结果

B.1. 延迟分析

在有 $W=128$ 个工作器和 $N \approx 70$ 条错误经验时，所有分析器只需一轮并行执行即可完成。当合并批大小为 $B_{\text{merge}}=32$ 时，层次化合并仅增加 $\lceil \log_{B_{\text{merge}}} N \rceil \approx 2$ 轮后续顺序执行，每个合并层一轮，因此总共需要 ≈ 3 轮顺序 LLM 调用。顺序基线分别需要 N 和 $\lceil N/B \rceil$ 轮，因为每次技能编辑都依赖前一次编辑。实际而言，并行整合耗时 3 分钟，Seq- $B=1$ 耗时 60 分钟（慢 20 $\times$ ），Seq- $B=4$ 耗时 15 分钟（慢 5 $\times$ ），且差距随 N 线性扩大。所有时间均为在相同 8-GPU A100 节点配置上测得的技能生成墙钟时间。

Table 7 | 表 1 中主要电子表格结果在随机种子 41、42 和 43 下的标准差。参考行给出绝对分数的标准差；演化行给出 表 1 所定义成对增量的标准差。汇总指标 Avg 稳定（每行标准差均 ≤ 3.4 ），且 表 1 的平均增益相对于单元格离散程度仍然很大，因此比较结果稳健。个别列（Vrf 与 OOD WikiTQ/HiTab 迁移）方差较大符合预期，并非不稳定的迹象：每个分数都汇总了通常超过 30 轮环境交互的长智能体执行，因此单项基准的运行间随机性会累积，但在 Avg 中被平均。为简洁起见，省略其他结果的标准差表；电子表格任务的标准差范围与本表相近。DAPO-Test、AIME’26 和 DocVQA ANLS 的标准差范围分别为 1.1–2.4、1.8–3.4 和 0.034–0.139。

Condition	Skill User: Qwen3.5-122B-A10B					Skill User: Qwen3.5-35B-A3B					
	SpreadsheetBench			OOD		SpreadsheetBench			OOD		Avg↑
	Vrf↑	Soft↑	Hard↑	WikiTQ↑	HiTab↑	Vrf↑	Soft↑	Hard↑	WikiTQ↑	HiTab↑	
Baseline (absolute scores)											
No Skill	3.75	0.90	0.70	0.81	0.67	1.80	2.77	1.25	1.27	5.28	0.42
Human-Written	8.10	1.39	0.40	0.65	0.71	2.93	0.57	0.40	0.36	2.68	0.46
Parametric	0.58	1.04	0.75	6.52	8.93	1.44	1.56	0.50	13.32	18.31	3.28
Skill Author: Qwen3.5-122B-A10B											
深化（相对人工编写初始技能的增量）											
+Error	4.82	1.10	0.78	0.78	0.60	6.76	3.36	2.06	2.62	4.16	1.02
+Success	12.53	0.75	1.35	1.68	0.75	3.69	3.78	2.23	1.43	4.01	1.43
+Combined	9.53	1.68	0.61	1.22	0.85	5.92	2.92	1.73	2.53	2.74	1.57
创建（相对参数化初始技能的增量）											
+Error	1.53	1.10	0.55	7.07	6.98	3.62	1.63	1.22	13.30	18.93	3.24
+Success	1.89	0.91	1.45	5.25	9.60	4.86	2.39	1.76	14.66	19.38	3.00
+Combined	3.04	0.90	1.10	6.07	9.69	3.04	1.48	1.13	13.52	19.72	2.99
Skill Author: Qwen3.5-35B-A3B											
深化（初始：人工编写）											
+Error	11.50	1.31	2.31	1.40	0.63	5.11	3.00	1.12	0.67	1.46	1.10
+Success	15.81	4.63	2.42	1.69	0.84	0.76	2.47	1.16	1.18	2.90	1.32
+Combined	9.00	2.14	1.33	1.94	0.85	0.50	3.62	2.54	3.27	2.71	0.88
创建（初始：参数化）											
+Error	2.18	1.04	0.58	6.96	8.84	1.76	1.77	1.08	14.60	17.83	3.36
+Success	1.61	1.00	0.90	6.94	8.64	4.51	1.10	1.19	14.24	19.02	3.21
+Combined	2.75	1.02	1.28	6.37	8.19	5.51	1.62	1.91	10.98	19.64	3.16

B.2. 数学推理

表 8 and 9 将主要分析中的消融实验扩展到数学推理。我们在这些数学消融实验中采用 +Error 设置，因为它是 表 2 中最强的数学设置。并行整合在所有报告的数学指标上均达到最佳或并列最佳，同时技能生成的实际耗时也最低。

Table 8 | 数学推理上的并行整合与顺序编辑对比（轨迹预算相同）。D-Test/AIME 的定义同表 2（122B 与 35B 使用者）。

Condition	Runner: 122B		Runner: 35B		Time↓
	D-Test↑	AIME↑	D-Test↑	AIME↑	
No Skill	92.0	90.4	89.0	83.3	0
Seq-B=4	93.0	91.7	89.0	68.3	3.8 min
Seq-B=1	94.0	85.8	90.0	81.7	25.9 min
Parallel	95.0	91.7	94.0	88.3	2.0 min

Table 10 | 使用 +Combined 轨迹池时, DocVQA 上的并行整合与顺序编辑对比。ANLS/Acc 的定义同表 3。

Setting	Runner: 122B		
	ANLS↑	Acc↑	Time↓
No Skill	0.6300	70.27	0
Seq-B=4	0.8674	90.80	5.2 min
Seq-B=1	0.8694	91.05	35.9 min
Parallel	0.8833	92.52	4.8 min

Table 9 | 在数学推理上, Trace2Skill 优于 ReasoningBank 检索。D-Test/AIME 的定义同表 2。

Condition	Runner: 122B		Runner: 35B	
	D-Test↑	AIME↑	D-Test↑	AIME↑
No Skill	92.0	90.4	89.0	83.3
ReasoningBank (error-only)	94.0	90.8	91.0	80.8
ReasoningBank (combined)	94.0	90.4	91.0	80.4
Trace2Skill +Error (ours)	95.0	91.7	94.0	88.3

Table 11 | 在 DocVQA 上, Trace2Skill 优于 ReasoningBank 检索。ANLS/Acc 的定义同表 3。

Setting	Runner: 122B		Runner: 35B	
	ANLS↑	Acc↑	ANLS↑	Acc↑
No Skill	0.6300	70.27	0.6582	73.17
ReasoningBank (combined)	0.8668	90.90	0.8568	89.62
Trace2Skill +Combined (ours)	0.8833	92.52	0.8740	92.00

B.3. DocVQA

表 10 and 11 将主文 DocVQA 结果扩展到顺序和检索记忆比较。我们在这些 DocVQA 消融实验中使用 +Combined 设置, 因为它是表 3 中最强的 DocVQA 设置。表 10 分离了 122B 组合协议的批次消融实验, 其中所有非基线行都使用相同的 25 条失败轨迹和 25 条成功轨迹, 只在顺序批大小与并行整合方面存在差异。这 50 个 DocVQA 样例仅用于演化, 并从评估中排除; 所有 DocVQA 分数均在其余 5,299 个留出样例上计算。如附录 B.1 所分析, 并行与顺序范式之间的效率差距将随轨迹数量增加而扩大。

B.4. 跨模型轨迹归纳

图 6 展示了这一跨模型轨迹归纳设置。使用 Qwen 轨迹时, 由 Gemma-4-31B-it 和 GPT-5.5-high 编写的 Deepening 技能, 相较于 No Skill (27.67% Vrf) 和 Human-Written (48.33% Vrf), 均能提升 Qwen3.5-122B 使用者的表现。最强设置是由 GPT-5.5-high 编写的 Deepening +Combined, Vrf 达到 68.00%, 表明跨模型轨迹证据仍能归纳出有意义的可移植技能。

实现细节。我们使用 vLLM (Kwon et al., 2023) 部署 Gemma-4-31B-it, 并采用其 Hugging Face 模型卡 (google/gemma-4-31B-it) 中官方推荐的生成配置; 在 ReAct 智能体运行中禁用推理。GPT-5.5-high 通过官方 OpenAI API 调用, 仅将推理强度设为 high, 其余设置均保持默认值。

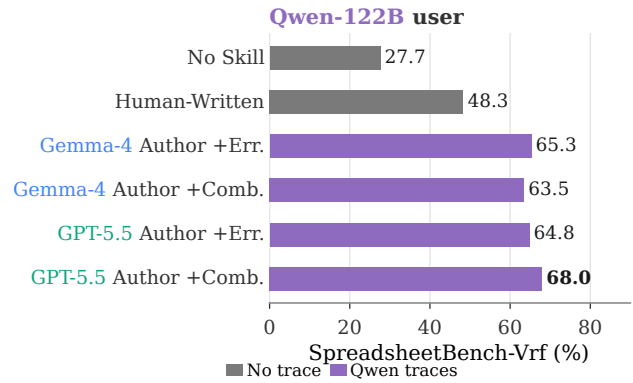


Figure 6 | SpreadsheetBench-Vrf 上的跨模型轨迹归纳 (通过率, %)。Gemma-4 和 GPT-5.5-high 基于 Qwen 轨迹编写技能, 随后由 Qwen3.5-122B 用户运行这些技能。

C. 定性分析

C.1. 智能体式与 LLM 错误分析

我们对智能体式错误分析员 $\mathcal{A}^-$ 和单次调用的 +Error LLM 基线共同分析的 33 个错误案例进行定性审查。两条流水线仅在 4 个案例 (12.1%) 上达成高度一致, 而在 18 个案例 (54.5%) 上对根本原因或合适的技能补丁存在明显分歧。主要区别在于证据获取: $\mathcal{A}^-$ 可以检查输入/输出产物, 将提交的答案与标准答案比较, 并验证候选修复; 而 +Error LLM 必须仅从执行日志中推断失败原因。

这一局限使得仅使用 LLM 的分析器容易产生日志层面的假阳性。在出现解析错误消息的案例中, +Error LLM 在 57% 的案例中将解析错误归为主要根因, 而智能体式分析员的该比例为 14%。在一条典型轨迹中, 尽管产物评估表明输出已经正确, +Error LLM 仍幻觉出三种不同的失败原因。相比之下, 智能体循环可在检查生成文件并重新运行相关验证步骤后否定此类解释。

这些定性差异解释了为什么智能体式补丁在 § 4.1 中迁移得更可靠。 $\mathcal{A}^-$ 不会编码日志中的表层症状, 而是将补丁锚定到已验证的失败机制: 错误的目标范围、过期的公式值、损坏的工作簿结构、类型转换, 或缺失回读验证。由此产生的编辑更有可能成为领域通用的防护规则, 且在作为可移植技能应用时, 更不容易损害 ID、跨模型或 OOD 设置。

C.2. 补丁选择定性分析

补丁无关回退。后续迭代补丁确实会改变智能体在部分任务上的行为, 使其按照补丁意图由错误变为正确。然而, 相同编辑也会改变与目标失败无关的行为, 使其他此前正确的任务由正确变为错误。这些翻转大多发生在源任务之外: 在十次迭代的贪心 +Combined 路径 (种子 41) 中, 选中补丁的源任务与评估划分没有任何重叠, 因此某一步骤破坏的此前正确任务从未被选中补丁针对; 这属于副作用, 而非针对性回退。因此, 每个具体化的步骤都记录了两个方向上数量可观的翻转 (例如, 某一步有 57 个由失败变为通过, 而有 21 个由通过变为失败; 另一步则分别为 26 和 46), 所以每次添加都会同时修复和破坏任务。我们仔细检查了这些翻转, 发现大多数由失败变为通过的翻转与补丁相关, 而由通过变为失败的翻转通常无关。由于每个贪心步骤都会

同时修复和破坏任务，净准确率没有继续上升，而是进入平台期。因此，局部最优的单个补丁无法组合成全局增益：补丁价值取决于新增补丁之间如何交互，所以必须优化补丁的组合效果，而不是其孤立的边际贡献。

语义重叠。相同的失败模式会跨迭代反复出现，因此流水线会重复提出语义重叠、针对相同电子表格行为的补丁——包括公式重算、工作簿结构保留、引用整合和验证清单。贪心路径具体体现了这一点：在十次迭代中，选择器不断回到少数几个主题——重算与验证（第 1 和第 6 次迭代）、结构/表头/工作表处理（第 2 和第 9 次迭代），以及格式/类型/日期处理（第 4、5、7 和 8 次迭代）。从一个已被覆盖的主题中新选出的补丁，主要只是在重述技能中已有的指导，因而几乎不会带来边际增益。

C.3. 学习到的 SoP 细节

我们检查了 122B Deepening +Combined 运行生成的 323 个映射补丁。四种最常见的 SoP 分别被 323 个补丁中的 55.1%、54.8%、42.7% 和 16.4% 引用¹；这些占比之和超过 100%，因为单个补丁可以引用多个主题。

公式重算与写回验证（55.1% 的补丁）。每次写入公式后运行 `recalc.py`，并使用 `data_only=True` 重新打开文件以确认求值结果；跳过这一步会使单元格保留陈旧值，是该次运行中最常见的单一错误模式。

工具选择：使用 openpyxl 而非 pandas.to_excel()（54.8% 的补丁）。使用 pandas 进行读取/转换逻辑，使用 openpyxl 进行写回；首先将输入文件复制到输出路径，以保留所有结构锚点。pandas.to_excel() 会在无提示的情况下破坏公式关系和命名区域。

显式回读验证（42.7% 的补丁）。写入后重新打开输出文件，并在提交前确认每个目标单元格都保存了预期值；失败的错误轨迹通常缺少这一检查。

结构编辑安全性（16.4% 的补丁）。按降序删除行，以防止索引偏移导致损坏；编辑前复制输入工作簿，以保留格式和公式。错误轨迹记录了这两种失败模式，而成功轨迹证实了这一保护性工作流。

小众特性被分流至 references/。低支持度的观察结果不会被丢弃，而是被分流到 13 个补充参考文件中，而非主 SKILL.md。例如，特殊业务逻辑下的单元格颜色提取以及 FIFO 与 LIFO 不匹配问题，会被放入按需参考资料。这与成熟的技能设计实践一致：过程性指导从一般规则逐步深入到具体情形，主文档编码通用工作流规则，而 references/ 则作为低频边缘情况的按需查阅层。Trace2Skill 无需人工整理，而是从轨迹证据中自动恢复这一层次结构。

其他中等支持度的 SoP。以下 SoP 在同一次运行中具有中等支持度（约占补丁的 3.1%–4.6%），也被编码进演化后的技能中。

目标区域与答案位置验证（4.6% 的补丁）。写入前，根据任务元数据核实准确的目标工作表名称、单元格区域和 `answer_position` 字段。误读这些字段——写入错误的工作表或存在一位偏差的区域——会造成静默失败：不产生错误消息，但得分为零。

数据类型与日期时间保留（4.6% 的补丁）。将日期和数值写为 Python 原生类型，而非字符串。无论是 pandas 日期解析还是 openpyxl 单元格赋值，都可能在无提示的情况下将日期时间值字符串化；写入前检查每列的 dtype，并使用 openpyxl 的原生日期时间赋值。

编辑前探索工作簿结构（以成功为主，~4.0% 的补丁）。在任何写入操作之前，列出所有工作表、检查行列布局并核实表头位置。这种编辑前探索可以防止工作表错误和区域错误，在该次运行的 151 个偏向成功的补丁

¹Trace2Skill 的合并过程会追踪所归纳出的合并补丁（SoP）源自哪些原始补丁。

Table 12 | 在技能深化、技能创建以及全部 122B/35B 作者—使用者组合上，对比智能体式错误分析 (+Error) 与单次调用的 +Error LLM。Vrf/Soft/Hard、OOD 划分 (WikiTQ/HiTab) 和 Avg 的定义同表 1。每列中较优的方法以**粗体**标出。

Condition	Skill User: Qwen3.5-122B-A10B					Skill User: Qwen3.5-35B-A3B					Avg
	SpreadsheetBench			OOD		SpreadsheetBench			OOD		
	Vrf	Soft	Hard	WikiTQ	HiTab	Vrf	Soft	Hard	WikiTQ	HiTab	
Skill Author: Qwen3.5-122B-A10B											
深化											
+Error (ours)	65.83	46.60	27.43	76.30	43.45	36.67	22.47	6.23	18.28	14.85	36.21
+Error LLM	67.00	43.93	25.23	39.81	36.68	25.00	22.43	6.23	11.24	9.74	28.00
创建											
+Error (ours)	49.00	40.37	23.37	31.62	15.66	28.83	23.23	7.87	22.20	23.58	26.02
+Error LLM	27.17	27.73	16.20	47.26	30.26	19.83	17.60	4.70	23.30	36.53	26.60
Skill Author: Qwen3.5-35B-A3B											
深化											
+Error (ours)	65.00	44.80	25.17	68.32	38.93	27.00	22.20	8.20	11.73	13.38	32.58
+Error LLM	37.83	22.93	12.83	77.05	42.05	30.50	20.17	8.73	9.95	12.73	28.80
创建											
+Error (ours)	27.17	28.90	18.53	81.38	45.61	24.00	21.00	6.53	32.80	31.75	34.45
+Error LLM	22.00	27.67	16.60	54.61	37.93	23.50	16.87	4.93	11.24	33.75	26.49

中占有相当大的比重。

D. 完整的智能体式错误分析结果

表 12 将正文中的平均值比较扩展为逐数据集结果，覆盖两种技能编写者、两种演化模式和两种技能使用模型。主要规律是，智能体式错误分析更为稳定：+Error（我们的方法）在四个“编写者-模式”组合中的三个上取得更高的平均值，并在 SpreadsheetBench 的大多数列上胜出。唯一的例外是由 122B 编写的 Creation，其中 +Error LLM 的平均值略高，因为它在 OOD 表格基准上表现更好；即便如此，智能体分析器在分布内 SpreadsheetBench 指标上仍然更强。总体而言，完整表格支持正文中的核心主张：与仅分析日志相比，检查产物并验证修复所产生的错误补丁具有更可靠的迁移能力。

E. 选择性补丁聚合细节

主要实验在层次整合后应用所有已学习的补丁；选择性聚合则进一步追问，某个补丁子集是否能产生更好的技能。我们复用 § 2.1 的形式化： $\mathcal{S}$ 是技能， π_θ 是固定智能体， $\mathcal{P}$ 是 Stage 2 (§ 2.3) 产生的轨迹级补丁池， $c(p)$ 是支持补丁 $p \in \mathcal{P}$ 的源轨迹数量。我们用 $\mathcal{S} \oplus \mathcal{P}'$ 表示通过 Stage 3 差异应用 (§ 2.4) 将补丁子集 $\mathcal{P}' \subseteq \mathcal{P}$ 应用于 $\mathcal{S}$ 后得到的技能，用 $\text{acc}_V(\mathcal{S}) \triangleq \mathcal{P}(\mathcal{S}; \pi_\theta, V)$ 表示其在 32 个任务的验证集 V 上的验证通过率。下面的两个选择器都会搜索一个子集，使得其实例化技能最大化 acc_V 。

贪心 top-1 选择。在第 t 次迭代中, 我们评估当前技能 S_t , 收集新提出的补丁 $\mathcal{P}_t$, 并从中选取覆盖度最高且满足 $c(p) \geq 5$ 的五个补丁, 形成候选集 C_t 。对于每个 $p \in C_t$, 我们通过添加单个补丁和移除单个补丁的干预来估计其贡献:

$$\begin{aligned}\Delta^+(p) &= \text{acc}_V(S_t \oplus p) - \text{acc}_V(S_t), \\ \Delta^-(p) &= \text{acc}_V(S_t \oplus \mathcal{P}_t) \\ &\quad - \text{acc}_V(S_t \oplus (\mathcal{P}_t \setminus \{p\})).\end{aligned}$$

丢弃满足 $\Delta^+(p) < 0$ 或 $\Delta^-(p) < 0$ 的补丁; 对余下补丁按 $r(p) = \Delta^+(p) + \Delta^-(p)$ 排序, 并以覆盖度作为平局决胜条件, 对排名最高的补丁 p_t^* , 下一个技能为 $S_{t+1} = S_t \oplus p_t^*$ 。§ 4.3 报告了 SpreadsheetBench-Vrf 结果。

补丁子集上的贝叶斯优化。BO 从 Stage-2 补丁池 (§ 2.3) 中固定一个补丁全集 $\mathcal{P} = \{p_j\}_{j=1}^m$, 保留覆盖度最高的 $m \leq 15$ 个补丁。它搜索二进制包含向量 $x \in \{0, 1\}^m$, 用 $\mathcal{P}_x = \{p_j : x_j = 1\}$ 表示选定子集, 并从初始技能 S_0 出发。目标为

$$f(x) = \text{acc}_V(S_0 \oplus \mathcal{P}_x) - \lambda \|x\|_0, \quad \lambda = 0.$$

初始设计会评估空子集、所有单例子集、五个随机混合子集和完整子集。每个批次后, 将已评估子集分为良好集 G 和不良集 B ; 前者包含按 $f(x)$ 排名前 $\gamma = 0.2$ 比例的子集, 后者包含其余子集。在 Beta 平滑 $\alpha = 1$ 下, 每个补丁的伯努利包含率为

$$\mu_j^G = \frac{\sum_{x \in G} x_j + \alpha}{|G| + 2\alpha}, \quad \mu_j^B = \frac{\sum_{x \in B} x_j + \alpha}{|B| + 2\alpha}.$$

候选子集从 $\phi_j = (1 - \rho)\mu_j^G + \rho/2$ 中使用 $\rho = 0.1$ 采样, 并按 TPE 采集函数排序

$$A(x) = \sum_{j=1}^m [\log q(x_j; \mu_j^G) - \log q(x_j; \mu_j^B)], \quad q(b; \mu) = \mu^b (1 - \mu)^{1-b}.$$

我们每轮从一个包含 500 个候选项的池中评估排名最高的八个候选项, 在四轮后或连续三轮无改进时停止, 并返回观测到的最佳子集。§ 4.3 将 BO 选择的 +Error 技能与应用全部 +Error 补丁进行了比较。

F. 更广泛的应用细节

我们将 Trace2Skill 扩展到三种更广泛的文档智能体场景: PDF 提取、PPTX 编辑和 DOCX 编辑。所有设置均以 Anthropic 的官方文档技能作为底层技能家族 (Anthropic, 2026a)。源轨迹仅用于技能演化, 而留出的目标任务仅用于评估。每项评估都使用该任务确切的本地验证器, 并报告与主要实验相同的任务级通过率指标。表 13 总结了正文中各模态采用的迁移设置; “Base” 表示应用所列源轨迹之前的直接前置技能。

PDF 提取。PDF 研究将 VRDU 和 VAREX 改造为本地 PDF 到 JSON 任务 (Wang et al., 2023b; Barzelay et al., 2026)。我们使用 VRDU Registration Forms 作为源语料库, 因为它提供了一组连贯的真实、视觉信息丰富的表单提取轨迹, 其中反复出现相似的字段与布局惯例。我们在单独留出的结构化提取基准 VAREX Flat 上评估, 以检验这些归纳出的表单提取规程能否跨数据集迁移, 而非仅迁移到同一语料库的更多样例上。报告的 PDF 分数使用经验证最强的 VRDU Registration 源划分; 其他 Registration 划分也带来正向收益, 但增益较小。

PPTX 编辑。PPTX 研究使用 Talk-to-Your-Slides 中的 TSBench 演示文稿编辑任务 (Jung et al., 2026)。PPTX 结果从一组幻灯片上的 TSBench 训练轨迹进行演化, 并在由留出幻灯片组成、与训练幻灯片互斥的 TSBench OOD 划分上评估, 从而检验跨演示文稿文件的迁移, 而非同一批幻灯片上的更多编辑。

领域	源轨迹	留出评估	Base	Trace2Skill	增益
PDF	VRDU Registration Forms	VAREX Flat 验证集	76.9%	85.3%	+8.4 pp
PPTX	TSBench 训练轨迹	TSBench 幻灯片互斥 OOD	72.5%	88.8%	+16.3 pp
DOCX	生成的文档操作任务	OfficeBench DOCX 留出集	79.7%	87.5%	+7.8 pp

Table 13 | 更广泛的文档智能体迁移。每一行都从源轨迹演化一项 Anthropic 官方文档技能，并在留出的目标领域上评估。Base 和 Trace2Skill 表示目标通过率 (%)；增益为二者之差，以百分点 (pp) 计。

DOCX 编辑。DOCX 研究使用 OfficeBench 作为留出的目标集，因为它包含具有可验证 Word/DOCX 输出的真实办公自动化任务 (Wang et al., 2024a)。我们将全部 64 个可转换的 OfficeBench DOCX 子任务仅用于评估，并设置 max_turns=100。由于留出该目标集后，剩余真实且可验证的 DOCX 智能体数据过少，我们使用生成的办公文档轨迹作为源分布。这些轨迹涵盖编辑模板、保留文档结构、跨文件追加内容以及生成所需附属文件等可复用操作。

G. 与同期技能演化系统的正面对比

本附录是对 § 4.1 中同等条件比较的正面对比补充：它让 Trace2Skill 与三个完整的同期技能演化系统——XSkill (Jiang et al., 2026)、EvoSkill (Alzubi et al., 2026) 和 SkillGen (Ma et al., 2026)——直接竞争，而不是每次只剥离一个设计选择。我们基于三个原因未将其放入正文。**归因：**完整系统的分数会将设计思想与其基础模型、执行框架及工程实现混杂在一起，因而无法说明差异由哪个因素驱动；相比之下，§ 4.1 在共享轨迹池、模型和执行框架下只改变单一设计选择，这正是我们提出因果主张的依据。**范围：**这些系统追求不同目标，并围绕特定的（通常是专有的）基础模型、定制执行框架及不同任务领域构建；而我们有意研究可自我演化的开放模型。将它们移植到同一基准只能说明其在我们的设置中的表现，而不能代表其设计的性能上限，因此我们对这些数字作保守解读。

我们依照其官方代码库复现这些流程，并将其适配到统一协议：使用由 vLLM 部署的同一开放基础模型 Qwen3.5-122B-A10B，在 SpreadsheetBench-Verified 的 0:200 切片上演化，并在留出的 200:400 切片上使用官方实例准确率（即我们的 Vrf 指标）进行评估。图 7 报告了相对于 No Skill 下限、多个系统同样采用的 Human-Written 技能，以及 Trace2Skill Deepening +Error/+Combined 的结果。Trace2Skill 取得最高 Vrf (+Error 为 65.83, +Combined 为 69.83)，超过全部三个系统；各系统的具体设置及其相对原生流程的改动详述如下。

XSkill。XSkill 是一种非参数记忆方法，它累积任务级过程技能和动作级经验，然后在推理时检索、适配并注入这些内容；该方法为多模态工具使用智能体而设计。我们在 0:200 切片上收集 200 条轨迹（与 Trace2Skill 使用相同的演化集），将逐轨迹技能合并成一个包含 865 个词的全局 SKILL.md（采用批量 LLM 合并），并提炼出 20 条精选动作级经验。测试时，我们针对每个实例按词法重合度检索前 3 条经验，让 LLM 根据任务适配全局技能，将其注入系统提示词，并让智能体最多运行 100 轮。该复现达到 23.0 Vrf。我们还用 Qwen3-Embedding-0.6B 替换了词法检索器，但其表现略差，为 20.0。

EvoSkill。EvoSkill 是一个自我演化循环——由基础智能体、失败提议器、技能生成器、验证评估器和有界前沿组成——为搭配 Opus 4.5 的 Claude Code 执行框架而设计。我们在 0:160 上挖掘失败，在 160:200 上验证候选，并在 200:400（我们的 SpreadsheetBench Vrf 测试子集）上评估所选程序，前沿大小为 3；每次迭代都会提出并生成一个替代的 xlsx/SKILL.md，然后在验证切片上评分，模型使用由 vLLM 部署的 Qwen3.5-122B-A10B。我们运行两种执行框架：与我们设置一致的 ReAct 风格本地执行框架达到 59.5 Vrf (5 次迭代)，

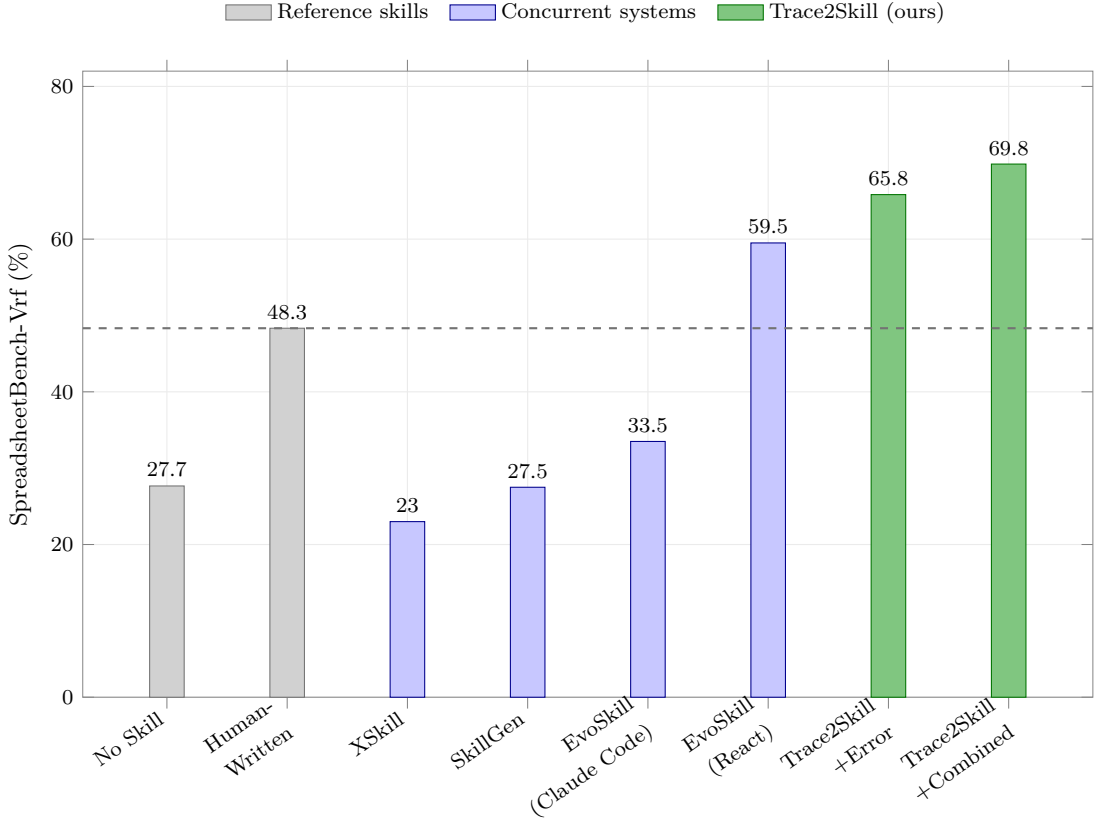


Figure 7 | 在 SpreadsheetBench-Verified (Vrf, 通过率%) 上的一对一比较, 所有系统均使用相同的开源基础模型 (Qwen3.5-122B-A10B) 和留出的 200:400 测试切片。柱状图包括 No Skill 下限、Human-Written 初始技能 (虚线)、三个复现的同期系统, 以及 Trace2Skill Deepening。EvoSkill 的两根柱对应同一个忠实复现, 唯一差别是智能体框架 (Claude Code, 遵循原论文; 与 React, 匹配我们的设置)。

而通过 Anthropic 兼容代理访问的 Claude Code (最接近原生执行框架) 达到 33.5 (8 个工作器, 在接近第 11 次迭代时提前停止)。

SkillGen。SkillGen 通过配对验证门后的对比式失败/成功分析归纳紧凑技能, 再进行留出评估。我们在 0:200 上收集基线轨迹, 使用 Qwen3-Embedding-0.6B 对失败与成功摘要进行聚类 (k -means), 以综合生成对比观察, 随后生成紧凑的 SKILL.md, 并在 0:200 上通过净增益接受门进行验证 (我们使用 $\min_net_gain_abs=1$; 上游默认值为 ≥ 3), 最多进行 8 轮优化, 最后在 200:400 上评估被接受的技能。训练切片上的基线为 52.5, 第 1 轮净增益为 0 (拒绝), 第 2 轮为 -4 (拒绝), 第 3 轮为 +1 (接受), 在留出切片上取得 27.5 Vrf。训练集上的提升与测试集上的退步可能源于分布偏移: 训练切片混合了 125 个工作表级任务和 75 个单元格级任务, 而测试切片仅包含单元格级任务。

这些正面对比是对 § 4.1 中无混杂、同等条件比较的补充, 而非替代; 我们的主张以后一类比较为基础。

H. Skill-Creator 基线

对于仅评估 Vrf 的外部基线, 我们通过使用 Opus 4.6 medium 的 Claude Code, 使用 Anthropic 官方 skill-creator 技能进行技能起草和改进 (Anthropic, 2026c)。表 14 报告了主要电子表格评估中相同的技能用户/轨迹来源配对在 SpreadsheetBench-Verified 上的 Vrf 分数, 同时给出相应的表 1 基线和 Trace2Skill +Error 结

用户/轨迹来源	Deepening			Creation		
	基础	skill-creator	Trace2Skill +Error	基础	skill-creator	Trace2Skill +Error
122B/122B	48.33	27.33	65.83	26.17	26.67	49.00
122B/35B	48.33	19.50	65.00	26.17	18.33	27.17
35B/35B	9.67	18.50	27.00	20.17	17.67	24.00
35B/122B	9.67	28.00	36.67	20.17	27.33	28.83
平均	29.00	23.33	48.63	23.17	22.50	32.25

Table 14 | SpreadsheetBench-Verified (Vrf) 上与 Anthropic skill-creator 基线的比较 (通过率, %)。对于每个技能用户/轨迹来源配对, 分别在 Deepening 和 Creation 下将 skill-creator 与相应的 Base 技能及 Trace2Skill +Error 进行比较; 最后一行为各列的平均值。

果。在 Deepening 和 Creation 中, 所得技能平均而言都未能改善相应的基础技能, 而 Trace2Skill +Error 始终更强。

该基线使用的提示词如下:

Skill Creator 基线提示词: Deepening

I have an agent running spreadsheet jobs using an xlsx skill. The agent's traces are provided in /path/to/traces/, where error and success traces are annotated with *_FAILURE.md and *_SUCCESS.md. Your job is to deepen the xlsx (/path/to/xlsx) to improve the agent's future performance when using the new skill. You should first induce the common and generalizable failure and success patterns from the traces and patch the xlsx skill using the skill-creator skill.

Skill Creator 基线提示词: Creation

I have an agent running spreadsheet jobs. The agent's traces are provided in /path/to/traces/, where error and success traces are annotated with *_FAILURE.md and *_SUCCESS.md. Your job is to create a spreadsheet skill to improve the agent's future performance equipped with the skill. You should first induce the common and generalizable failure and success patterns from the traces and then create the skill using the skill-creator skill.

I. 提示词模板与中间输出

本附录复现了各流水线阶段使用的关键提示词模板, 并展示具有代表性的中间输出, 以使该流水线完全透明且可复现。

I.1. 阶段 1: 智能体系统提示词模板

智能体 π_θ 在轨迹收集期间使用以下系统提示。推理时, 技能 $\mathcal{S}_0$ 被前置到用户上下文中。请注意, 这与标准技能加载过程不同, 后者中智能体初始时只能访问技能描述。由于 Trace2Skill 专注于改进相对于任务已知的固定目标技能, 我们通过将 SKILL.md 内容预加载到系统提示中来简化此过程。因此, 不需要标准的技能选择步骤。重要的是, Trace2Skill 的技能使用智能体仍需要以程序化方式发现预加载 SKILL.md 中引用的资源 (例如资源和脚本), 这些内容并未预加载。

Stage 1 — 智能体系统提示（缩略）

角色：你是一名专家级 role（例如电子表格分析）智能体。

技能上下文：[在此插入 $\mathcal{S}_0$ 的内容]

任务：描述任务和输入文件。

可用工具：描述工具和环境。例如，用于可访问文件系统的 ReAct 的 bash（shell 执行）。

格式：ReAct 式交互 — 在推理轨迹与工具调用之间交替，直至完成任务。

I.2. Stage 2: 分析员提示模板与补丁示例

在阶段 2 中，提出补丁的智能体首先提取类似于 (Ouyang et al., 2026) 的错误与成功记忆条目，这些条目是可能有助于未来任务执行、可泛化的轨迹级知识。接下来，智能体读取原始技能目录，然后提出补丁，将这些记忆条目编码进技能中。

I.2.1. 错误分析器提示词 ($\mathcal{A}^-$)

错误分析器系统提示词（缩略版）

角色：你是一名处理 {domain} 任务的专家级失败分析智能体。

任务：给定智能体的执行产物（日志 + 生成文件）和真实解，诊断智能体为何失败，识别因果性失败原因，并通过实施一项能使智能体输出与真实解一致的最小修复来验证你的诊断。你的分析必须**系统化、以证据为驱动且可复现**。能够验证时，**不要猜测**。

必需工作流（强制）：

1. 理解任务和失败表面 — 准确识别输出中的错误。
2. 将失败追溯至智能体行为 — 定位产生不匹配的决策或代码步骤。
3. 用最小修复验证根本原因 — 写出修复后的输出，并根据真实解重新评估。
4. 重新评估 — 若仍然失败，返回步骤 1-3 并修订诊断。

输出：生成 (1) 失败原因条目 — 以可观察智能体行为为依据的系统化因果原因；(2) 失败记忆条目 (≤ 3) — 智能体为避免类似失败而应记住的可泛化见解。

I.2.2. 成功分析器提示词 ($\mathcal{A}^+$)

成功分析器系统提示词（缩略版）

角色：你是 AI 智能体系统成功模式分析方面的专家。

任务：给定一条成功的智能体轨迹，识别促成正确答案的**可泛化行为模式**。

要求：

广泛覆盖 — 轨迹中的每一种有效行为都必须由某个模式涵盖。

频率意识 — 覆盖更多实例的模式应优先列出；罕见行为应吸收到最接近的更广泛模式中。

泛化性 — 每个模式都必须描述一种一般机制，而不是单一的任务特定细节。

输出：一组紧凑的成功记忆条目，包含标题、描述，以及观察到的有效行为的具体示例。

I.2.3. 轨迹级补丁示例（错误分析器输出）

每个分析员针对单条轨迹生成一个结构化补丁。下面的示例来自错误分析员（ $\mathcal{A}^-$ ），它处理的一条失败轨迹中，智能体删除了指定范围之外的行。该补丁建议在 SKILL.md 中新增一节关于行操作安全的内容；这个补丁随后与另外 52 个引用相关结构编辑失败的补丁一起，被整合到“结构编辑安全”SoP (§ 4.2) 中。

Stage 2 轨迹级补丁示例（错误分析员，单条轨迹）

分析员推理：单条错误记录揭示了一种明确的失败模式：智能体删除了指定范围之外的行，且无法区分部分为空的数
据行与完全为空的填充行。该技能缺乏任何关于行/列操作或范围边界强制的指导。

```
--- a/SKILL.md
+++ b/SKILL.md
@@ -55,0 +55,26 @@ ## Formula Verification Checklist

+### Row and Column Operations
+
+#### Range Boundary Enforcement
+- When a task specifies a cell range (e.g., F14:M83), operations
+ MUST stay within those exact boundaries
+- Parse range format: start_col + start_row : end_col + end_row
+- Never apply operations to rows/columns outside the range
+
+#### Row Deletion Logic
+Before deleting any row, verify BOTH conditions:
+1. Target columns are empty (as specified by task)
+2. Row falls within the explicit deletion range
+
+**Do NOT delete** completely empty rows across ALL columns unless
+explicitly instructed -- these often serve as structural padding.
+
+#### Post-Operation Validation
+After row/column operations:
+- [ ] Count remaining rows matches expected count
+- [ ] No header/footer/padding rows outside range were affected
+- [ ] Data integrity is maintained in adjacent columns
+- [ ] Range boundaries were respected (no off-by-one errors)
```

I.3. Stage 3: 合并算子提示与整合补丁示例

I.3.1. 合并算子提示词 ($\mathcal{M}$)

合并算子系统提示词（完整版）

你是一名技能编辑协调器。你会收到多个独立提出的补丁，每个补丁都建议对技能文件夹进行修改。你的任务是它们合并为一个连贯且无冗余的补丁。

准则：

- 去重：**当多个补丁提出相同或高度相似的编辑时，保留最佳版本（最具体、措辞最佳）。
- 解决冲突：**若补丁对同一章节提出相互矛盾的编辑，选择依据更充分的一项，或综合二者形成更好的编辑。
- 保留独特见解：**不同补丁处理不同的失败 — 纳入所有独特且不重复的编辑。
- 保持简洁：**合并后的补丁应具有输入补丁中独特编辑总量的 $\leq$ 。删除冗余。
- 确保独立性：**合并补丁中的编辑必须在行级别相互独立 — 不同编辑不得以重叠行或同一文本段为目标，即使它们属于不同操作也不行。
- 原子化的创建/链接对：**针对 references/*.md 的 create 操作与在 SKILL.md 中插入其链接的编辑是不可分割的

一对 — 要么全部保留，要么全部丢弃。

常见模式偏置：当多个补丁独立提出处理同一类失败或成功模式的相似编辑时，将这种重复视为任务具有系统性的证据。优先保留此类常见编辑，并将其表述为一般原则，而不是实例特定的修复。

I.3.2. 最终整合补丁示例 p^* （完整合并层次之后）

以下摘录展示了在 SpreadsheetBench-Verified 的 323 个独立轨迹补丁上经过四层层次化合并后，最终整合补丁 p^* 的推理过程和代表性编辑。

阶段 3 最终补丁输出示例（摘录）

```
{
  "reasoning": "Merged 3 patches addressing mixed failure/success
evidence. Key consolidation decisions: (1) Synthesized recalc.py
workflow from all patches using the most prominent CRITICAL WARNING
placement, CSV fallback validation, and a verification loop;
(2) Consolidated library selection guidance into a comprehensive
decision tree; (3) Combined row deletion guidance emphasizing
bottom-up/right-to-left deletion order from all patches;
(4) Merged formula validation checklists without redundancy.",
  "edits": [
    {
      "file": "SKILL.md",
      "op": "insert_after",
      "target_section": "# Requirements for Outputs",
      "content": "### Important Automation Guidelines\n\n**Prefer Python over VBA for Automation**:\n\nWhen tasks request VBA macros or spreadsheet automation, implement the logic in Python using openpyxl/pandas instead. This provides better error handling, easier debugging, cross-platform compatibility, and avoids macro security issues."
    },
    {
      "file": "SKILL.md",
      "op": "insert_after",
      "target_section": "## Important Requirements",
      "content": "### When to Compute Directly in Python\n\nIf LibreOffice/recalc.py cannot evaluate certain formula patterns (complex array formulas, advanced INDEX/MATCH combinations), compute the logic directly in Python and write results as values. Use this fallback when formula returns errors after recalculation despite correct syntax."
    }
  ],
  "changelog_entries": [
    "Added automation guidelines preferring Python over VBA",
    "Consolidated recalc.py warnings into a single CRITICAL WARNING"
  ]
}
```

I.3.3. 翻译后差异示例：最终整合补丁 p^*

阶段 3 生成的 JSON 补丁 p^* 会以编程方式转换为统一差异格式，并直接应用到技能目录。以下摘录展示了 SpreadsheetBench-Verified 最终应用差异中的两个区块，分别对应两种最常见的 SoP：强制公式重算（55.1% 的补丁）和工具选择指导（54.8% 的补丁）。

阶段 3 应用差异示例（最终 p^* 的摘录）

```
--- a/SKILL.md
+++ b/SKILL.md
@@ -126,3 +261,18 @@ ## Common Workflow

## Common Workflow

+### CRITICAL WARNING: Formula Recalculation Is Mandatory
+
+**If you write ANY formulas to an Excel file using openpyxl,
+you MUST run recalc.py before considering the task complete.**
+
+Formulas written via openpyxl exist only as text strings until
+recalculated. Without running recalc.py:
+- Cells return None/empty when read with data_only=True
+- Evaluation fails even if formulas are syntactically correct
+- The output file is incomplete
+
+This is non-negotiable. Do not proceed to verification or
+delivery until recalc.py confirms success.
+
@@ -138,3 +285,9 @@ ### Working with openpyxl

+### Tool Selection Warning
+
+**CRITICAL**: When modifying spreadsheets that contain existing
+formulas you need to preserve:
+- Use openpyxl (load_workbook() then save()) -- formulas remain
+  as strings
+- Avoid pandas (to_excel()) -- silently converts formulas to
+  static values permanently
```